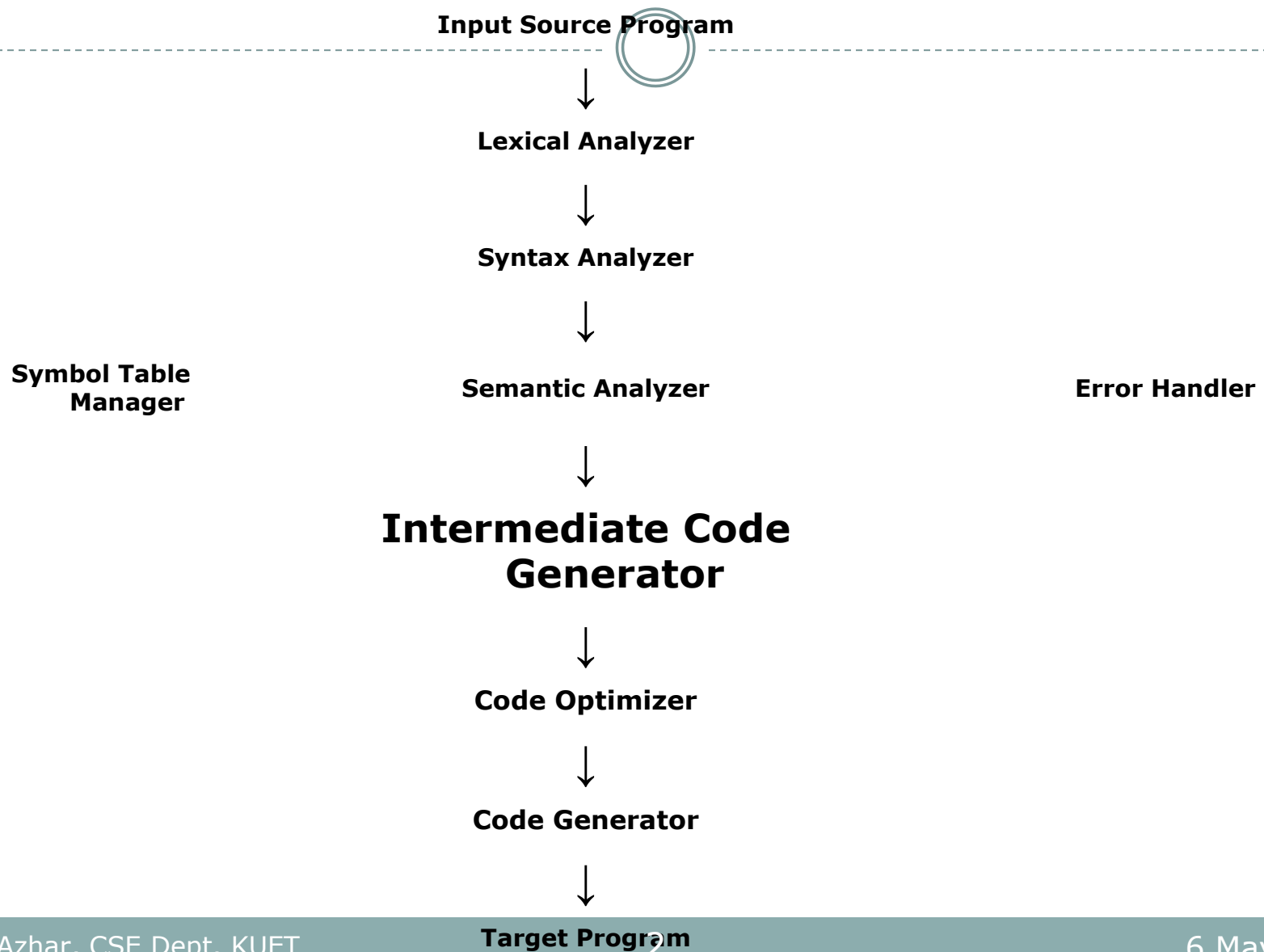


INTERMEDIATE CODE GENERATION



DR. K. M. AZHARUL HASAN
DEPT. OF CSE, KUET.

Phases of Compiler



Intermediate Code

3

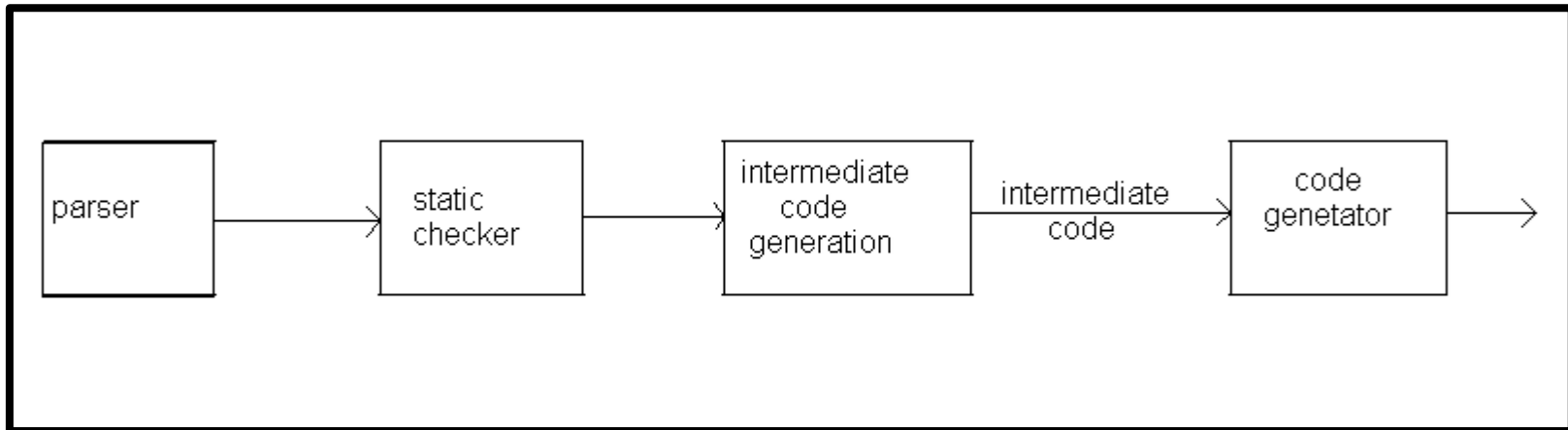
Defined as

The source code of a program is translated into a form called *intermediate code* which is suitable for code improvement and transformations before use to generate object or machine code for a target machine.

Intermediate Code Generation

4

Translate the source program Such as
declarations,
assignments and
flow-of-control statements
into an **intermediate form**



Intermediate Languages

5

Three types of intermediate representations

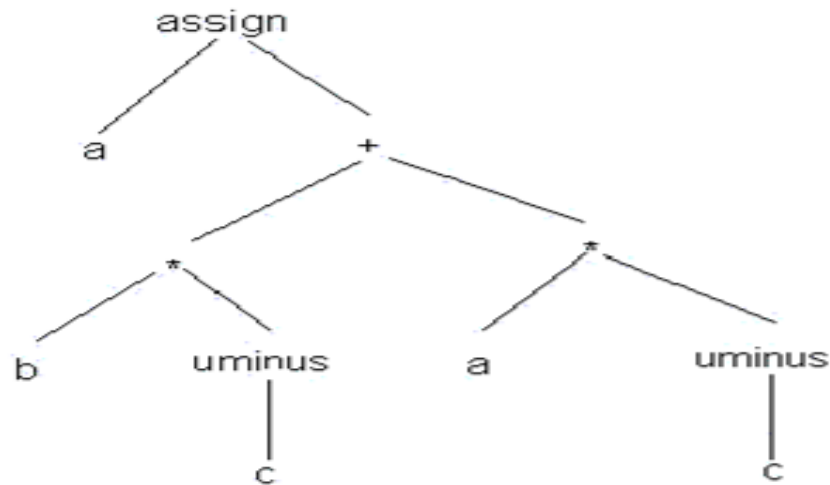
1. Syntax Trees
2. Postfix notation
- 3. Three Address Code**

Graphical Representations

6

A syntax tree depicts the natural **hierarchical structure** of a source program.

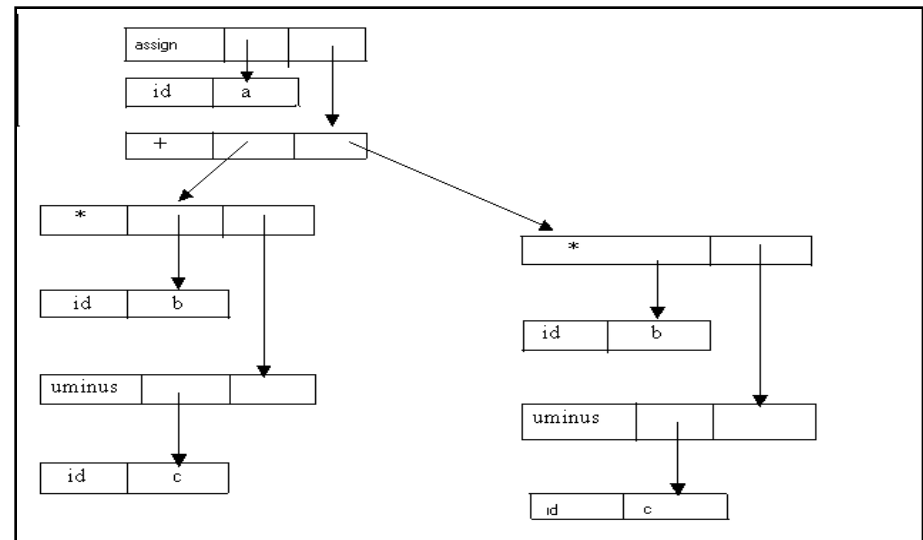
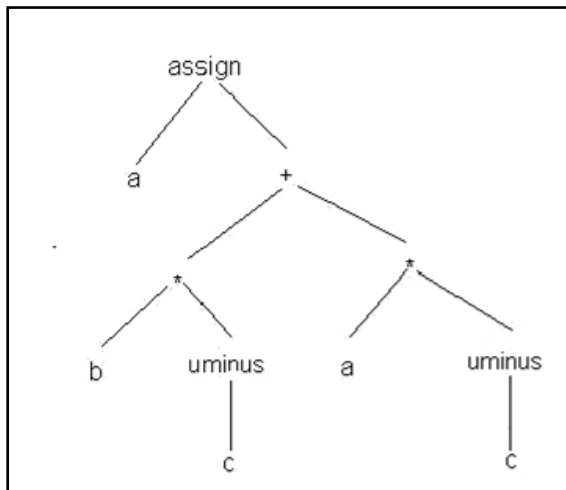
Ex. A syntax tree for the assignment statement
 $a := b * -c + a * -c$



Graphical Representations

7

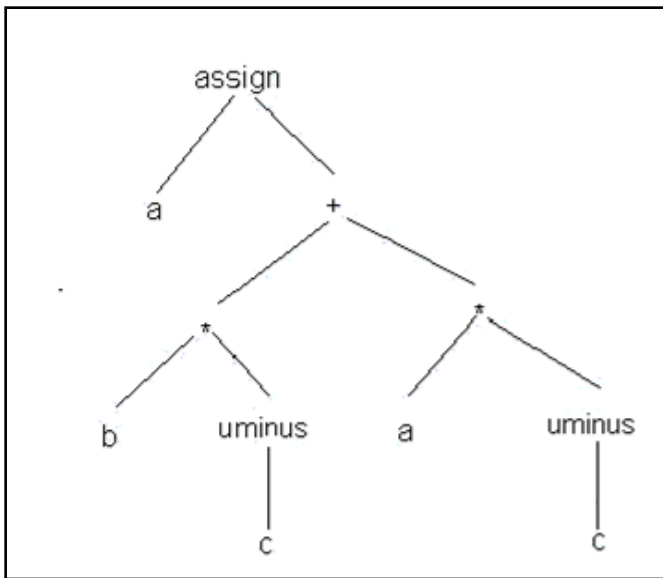
- There are **two representations** of the syntax tree.
 - Tree and array of records
- Each node is represented **as a record with a field for its operator** and additional fields for **pointers** to its children.
- All the nodes in the syntax tree **can be visited** by following **pointers**, starting from the root at position .



Graphical Representations

8

All nodes are allocated from an **array of records** and the index or position of the node serves as the **pointer to the node**.



0	id	b	
1	id	c	
2	uminus	1	
3	*	0	2
4	id	a	
5	id	c	
6	uminus	5	
7	*	4	6
8	+	3	7
9	id	a	
10	assign	9	8
11			

Three-Address Code

9

- A three-address statement is an abstract form of intermediate code.
- Three-address code is a sequence of statements of the **form**

$x := y \text{ op } z$

where x , y , and z are names, constants, or compiler-generated temporaries;

op stands for any operator, such as arithmetic operator, or a logical operator on Boolean-valued data.

- The reason for the term "three-address code" is that each statement usually **contains three addresses**, two for the operands and one for the result.

Three-Address Code(Cont...)

10

No arithmetic expressions are permitted, as there is **only one operator on the right** side of a statement

Thus expression like $\mathbf{x+y*z}$ might be translated into a sequence

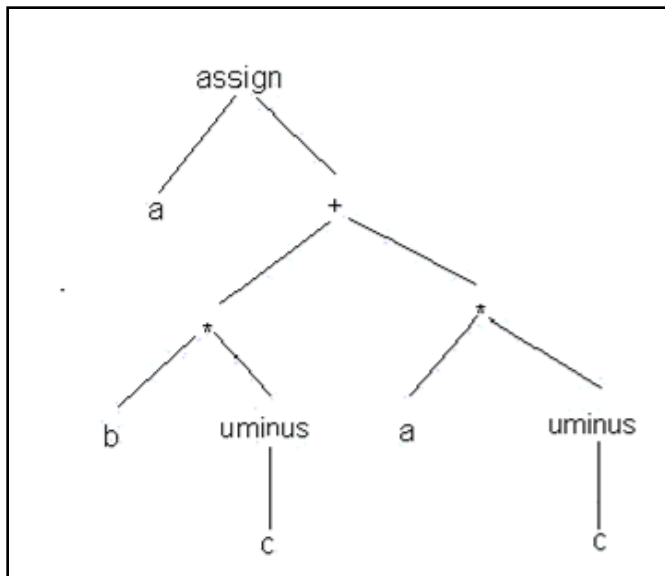
$t_1 := y * z$ where **t1** and **t2** are compiler-generated temporary names.
 $t_2 := x + t_1$

Three-Address Code(Cont...)

11

The use of names for the intermediate values computed by a program allow- three-address code to be easily rearranged

$$a := b * -c + a * -c$$



```
t1 := -c
t2 := b * t1
t3 := -c
t4 := a * t3
t5 := t2 + t4
a := t5
```

Three-Address Code : Types

12

- Assignment statements of the form $x := y \text{ op } z$, where op is a binary arithmetic or logical operation.
- Assignment instructions of the form $x := \text{op } y$, where op is a unary operation.

Unary operations include

unary minus, logical negation, shift operators, and conversion operators.

Three-Address Code : Types

13

- Copy statements of the form **$x := y$** where the value of y is assigned to x .
- The unconditional jump **$\text{goto } L$** . The three-address statement with label L is the next to be executed.
- Conditional jumps such as **$\text{if } x \text{ relop } y \text{ goto } L$** . This instruction applies a relational operator ($<$, $=$, $>=$, etc.)

Three-Address Code : Types

14

- Indexed assignments of the form $x := y[i]$ and $x[i] := y$.
 - The statement $x[i] := y$ sets the contents of the location i units beyond x to the value of y .
- Address and pointer assignments of the form $x := \&y$, $x := *y$ and $*x := y$.

Implementations of three-Address Statements

15

- ❑ Three address statements can be implemented as records with fields for the operator and the operands.
- ❑ Three such representations are
 - quadruples
 - triples
 - indirect triples.

Implementations of three-Address Statements

16

Quadruples

- A quadruple is a **record** structure with **four fields**
 - op, arg1, arg2, and result.
 - The op field contains an internal code for the operator.
 - The three-address statement **$x := y \text{ op } z$** is represented by placing y in arg1, z in arg2 and x in result.
 - Statements with unary operators like **$x := -y$** or **$x := y$** do **not use arg 2**.

Implementations of three-Address Statements

17

Quadruples

Operators like **param** use **neither arg2 nor result**.

Conditional and unconditional jumps put the **target label in result**.

```
t1 := -c
t2 := b * t1
t3 := -c
t4 := a * t3
t5 := t2 + t4
a := t5
```



	op	Arg1	Arg2	Result
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	a	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		a

Implementations of three-Address Statements

18

Triples

- To **avoid entering temporary names** into the symbol table. refer to a temporary value by the **position of the statement**
- three-address statements can be represented by records with only three fields: **op, arg1 and arg2.**
- The fields arg1 and arg2, for the arguments of op, are **either pointers to the symbol table or pointers into the triple structure.**
- Since **three fields** are used, this intermediate code format is known as **triples.**

Implementations of three-Address Statements

	op	Arg1	Arg2	Result
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	a	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		a

Quadruples

t1 := -c
t2 := b * t1
t3 := -c
t4 := a * t3
t5 := t2 + t4
a := t5

	op	Arg1	Arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	a	(2)
(4)	+	(1)	(3)
(5)	:=	a	(4)

Triples

Implementations of three-Address Statements

20

A **ternary operation** like $x[i] := y$ requires two entries in the triple structure.

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	[] =	x	i
(1)	assign	(0)	y

(a) $x[i] := y$

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(0)	= []	y	i
(1)	assign	x	(0)

(b) $x := y[i]$

Implementations of three-Address Statements

21

Indirect Triples

- Listing pointers to triples, rather than listing the triples themselves.
- This implementation is naturally called indirect triples.
- Use an array statement to list pointers to triples in the desired order

	<i>statement</i>
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

	<i>op</i>	<i>arg 1</i>	<i>arg 2</i>
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	assign	a	(18)

Comparison of the Implementations

22

- Using the quadruple notation, a three- address statement a **temporary can immediately access the location** via the symbol table.
- In the triples declaration; **moving a statement** that defines a temporary value requires **to change all references** to that statement in the arg1 and arg2 arrays.
- This problem **makes triples difficult** to use in an optimizing compiler.

Comparison of the Implementations

23

- Indirect triples present **no such problem**. A statement can be moved by reordering the statement list.
- Indirect triples **can save some space** compared with quadruples **if the** same temporary value is used more than once.
- The reason is that two or more entries in the statement array can point to the same line of the op-arg1-arg2 structure.